```julia
begin
    using Oscar: AbstractAlgebra.SymmetricGroup, symmetric_group, factorial,
    binomial, divexact, AbstractAlgebra.AlternatingGroup, partition, num_partitions,
    multinomial, order, ZZ
    using Latexify
end
```

```
WARNING: Imported binding AbstractAlgebra.AlternatingGroup was undeclared at imp
ort time during import to workspace#2.
```

```
(Multivariate polynomial ring in 6 variables x, y, n, k, ..., C, [x, y, n, k, P, C])
   over integer ring
```

```julia
Z, (x, y, n, k, P, C ) = ZZ[:x, :y, :n, :k, :P, :C]

```

I. Fundamental Counting Principles

Every counting problem relies on one or more of these three basic rules:

1. **The Multiplication Rule:** If one task can be performed in m ways and a second task in n ways, then the number of ways to perform both tasks sequentially is $m \times n$.
2. **The Addition Rule:** If one task can be performed in m ways and a second task in n ways, and these tasks are mutually exclusive (cannot be performed at the same time), then the number of ways to perform either task is $m + n$.
3. **The Subtraction Rule (Inclusion-Exclusion for two sets):** If tasks are not mutually exclusive, we must subtract the overlap: $|A \cup B| = |A| + |B| - |A \cap B|$.

$$N = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \\ 10 \end{bmatrix}$$

$$K = 5$$

We begin by asking a simple question—“How many?”—that has at least 4 interpretations.

- A. Any discrete math course will contain a significant portion devoted

to combinatorics, the mathematics of counting.

```
.1 Does order matter?
    Yes -> 100000
    No -> 30240.0

.2 Can you repeat numbers?
    yes -> 2002.0

    No -> 252.0
```

II. Permutations (Ordering Matters)

A permutation is an arrangement of all or part of a set of objects, where the **order is important**.

- **Permutations of n distinct objects: $n!$**
- **k-permutations of n (selecting k from n and ordering them):** $P(n, k) = \frac{n!}{(n-k)!}$

30240

```
begin
    P_fn(n, k) = divexact(factorial(ZZ(n)), factorial(ZZ(n) - k))
    P_fn(10, 5)
end
```

III. Combinations (Order Doesn't Matter)

A combination is a selection of all or part of a set of objects, where the **order is irrelevant**.

- **k-combinations of n (the "n choose k" formula):** $\binom{n}{k} = C(n, k) = \frac{n!}{k!(n-k)!}$

30240

```
begin
    C_fn(n, k) = binomial(n,k)
        P_fn(10, 5)
end
```

IV. The Relationship Between $P(n, k)$ and $C(n, k)$

Mathematically, a permutation can be thought of as a two-step process:

1. Choose k objects (Combination).

2. Arrange those k objects (Factorial).

$$P(n, k) = C(n, k) \times k!$$

30240

```
begin
    P_fn_2(n, k) = C_fn(n,k) * factorial(5)
        P_fn(10, 5)
end
```

V. Permutations with Repetition

If we have n objects where n_1 are of type 1, n_2 are of type 2, ..., and n_k are of type k, the number of distinct arrangements is given by the multinomial coefficient:

$$\frac{n!}{n_1!n_2!\dots n_k!}$$

30240

```
begin
    PR_fn(n, k) = divexact(factorial(n), prod(factorial([1:k])))

    P_fn(10, 5)
end
```

30240

```
multinomial(10, [5])
```

The number of 5-card poker hands is: $2598960 = 2,598,960$

```
Markdown.parse("""
            The number of 5-card poker hands is:
            $(latexify("$(binomial(52, 5)) = 2,598,960"))
            """)
```

VI. Implementation Tasks (Manual)

1. Implement a robust `factorial` function (or use `Oscar.factorial`).
2. Define `permutations(n, k)` and `combinations(n, k)`.
3. Create a visual representation of Pascal's Identity: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$.
4. Test the Multiplication Rule with a Cartesian product implementation.

120

```
begin
    p_n = num_partitions(4)
    S4 = SymmetricGroup(BigInt(p_n))
    order(S4)
end
```

[(), (1,2), (1,3,2), (2,3), (1,2,3), (1,3), (1,4,3), (1,2,4,3), (2,4,3), (1,4,3,2), (1,2)(3,

```julia
collect(S4)
```

Oscar.Partition{Int64}: [5, 4, 4, 2]

```julia
begin
    seq = ZZ(2)^5
    part = partition([5, 4, 4, 2])
end
```

Oscar.Partition{Int64}: [5, 4, 4, 2]

```julia
partition(5, 4, 4, 2)
```

[18.6]

```julia
begin
    Pascal_id(n, k) = [n - 1 / k - 1] + [n - 1 / k]
    Pascal_id(10, 5)
end
```

`$P(length(N), 5) = \frac{|S_{length(N)}|}{|S_{5}|} = 30240$`

```julia
begin

    # Instantiate the structural groups via GAP
    G_n = symmetric_group(length(N))
    G_n_minus_k = symmetric_group(length(N) - K)

    # Compute P(n,k) purely by dividing the orders of the groups
    # We use divexact because group theory guarantees |S_{n-k}| divides |S_n|
    permutations = divexact(ZZ(order(G_n)), ZZ(order(G_n_minus_k)))

    Markdown.LaTeX("P($length(N), $K) = \\frac{|S_{$length(N)}|}{|S_{$(length(N)-K)}|} = $(permutations)")
end
```

Group Orders Evaluated:

- $$S_{10} = 3628800$$
- $$S_{14} = 87178291200$$

```
begin
    # Exact Integer Domain


    # Instantiate the Symmetric Groups via GAP
    G_ns       = symmetric_group(length(N))           # S_10: Symmetries of the whole pool
    G_k       = symmetric_group(K)            # S_5:  Symmetries of the chosen items
    G_n_min_k = symmetric_group(length(N) - K)        # S_5:  Symmetries of the unchosen items

    # For Multisets (Stars and Bars), the abstract space is (n + k - 1)
    G_stars   = symmetric_group(length(N) + K - 1)    # S_14: Symmetries of the total Multiset space
    G_n_min_1 = symmetric_group(length(N) - 1)        # S_9:  Symmetries of the "bars" (dividers)

    Markdown.parse("""
    **Group Orders Evaluated:**
    * $(latexify("S_10 = $(order(G_ns))"))
    * $(latexify("S_14 = $(order(G_stars))"))
    """)
end
```

Final Exact Group Quotients:

- **Sequences:** $N = 100000$
- **Permutations:** $P\left(10, 5\right) = 30240$
- **Combinations:** $C\left(10, 5\right) = 252$
- **Multisets:** $M\left(10, 5\right) = 2002$

```
begin
    # 1. Sequences (Order Matters, Reps Allowed)
    # This maps to the cardinality of the function space: |N|^|K|
    sequences = ZZ(length(N))^K

    # 2. Permutations (Order Matters, No Reps)
    # The index of the unchosen symmetry group inside the total symmetry group
    # [S_n : S_{n-k}] = |S_n| / |S_{n-k}|
    permutation = divexact(ZZ(order(G_ns)), ZZ(order(G_n_min_k)))

    # 3. Combinations (Order Doesn't Matter, No Reps)
    # The index of the Young Subgroup (S_{n-k} × S_k) inside S_n
    # |S_n| / (|S_{n-k}| * |S_k|)
    combinations = divexact(
        ZZ(order(G_ns)),
        ZZ(order(G_n_min_k)) * ZZ(order(G_k))
    )

    # 4. Multisets (Order Doesn't Matter, Reps Allowed)
    # The index of the Young Subgroup (S_{n-1} × S_k) inside the Stars & Bars group S_{n+k-1}
    # |S_{n+k-1}| / (|S_{n-1}| * |S_k|)
    multisets = divexact(
        ZZ(order(G_stars)),
        ZZ(order(G_n_min_1)) * ZZ(order(G_k))
    )

    # Utilizing Latexify parsing valid Julia syntax (Function calls and assignments)
    Markdown.parse("""
    ### Final Exact Group Quotients:
    * **Sequences:** $(latexify("N = $(sequences)"))
    * **Permutations:** $(latexify("P($(length(N)), $(K)) = $(permutation)"))
    * **Combinations:** $(latexify("C($(length(N)), $(K)) = $(combinations)"))
    * **Multisets:** $(latexify("M($(length(N)), $(K)) = $(multisets)"))
    """)
end
```